

CampusCircle

High-Performance Social Match
& Graph Analytics Platform

Kelompok Siang 11 · Graph Database Paradigm (Neo4j)

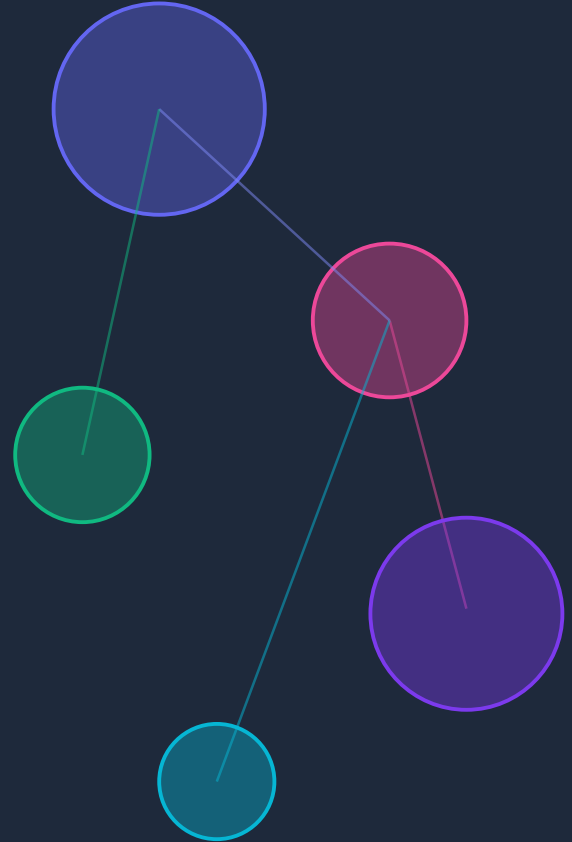
Raja Avicenna Al-Kindi Vijasa · 2406416352

Toriq Fathoni Dezi · 2406487115

Danish Al Fayyadh Sunarta · 2406416951

Alwahib Raffi Raihan · 2406397630

Perry Tjahya · 2406409965



Agenda Presentasi



01 Real-World Case Study

Hinge & Neo4j – dari masalah nyata ke solusi graph



02 Creative Application

CampusCircle – problem, motivasi, dan arsitektur



03 Data Modeling

Struktur graf vs relasional, query Cypher vs SQL



04 Benchmark & Hasil

Perbandingan performa Neo4j vs PostgreSQL



05 Lessons Learned

Trade-off, temuan, dan kontribusi individu

01 Real-World Case Study

Hinge & Neo4j: Belajar dari Dating App Terpopuler



Apa itu Hinge?

Hinge merupakan dating app relationship-oriented yang menggunakan algoritma swipe berbasis circle sosial.

Sangat populer di kalangan Milenial & Gen Z karena pendekatannya yang lebih otentik dan personal.

Keunikan Utama:

- Rekomendasi berbasis circle sosial
- Memanfaatkan mutual friends (derajat 2–3)
- Data relasi dari platform sosial (Facebook)
- Powered by iDating platform via Neo4j



Masalah yang Dihadapi

JOIN berlapis di SQL

Pencarian friends-of-friends (2–3 hop) membutuhkan JOIN yang sangat banyak dan kompleks

Performa menurun drastis

Semakin besar volume koneksi sosial, query relasional semakin lambat dan tidak scalable

Tidak real-time

Rekomendasi berbasis jaringan pertemanan tidak bisa dijalankan secara real-time di database relasional

Mengapa Neo4j?

Perbandingan SQL vs Graph Database untuk kasus sosial Hinge

Aspek	Relational DB (SQL)	Graph DB (Neo4j)
Traversal Multi-hop	JOIN berlapis, sangat lambat	Sangat cepat karena bebas index adjacency
Model Relasi	Foreign key & tabel pivot	Edge sebagai first-class citizen
Koneksi Derajat 3	Tidak efisien / hampir tidak mungkin	Native, tanpa trik tambahan
Query Sosial	Kompleks & lambat seiring skala	Cypher sederhana & cepat
Skalabilitas Graf	Menurun eksponensial	Tumbuh secara linear



Neo4j menggunakan Index-Free Adjacency: setiap node menyimpan pointer langsung ke tetangganya — traversal graf secepat pencarian pointer, bukan full table scan.

02

Creative Application



CampusCircle

Platform Social Matching Mahasiswa berbasis Graph Database



Discover

Temukan teman baru lintas fakultas berdasarkan mata kuliah & minat yang sama



Connect

Swipe & match dengan sesama mahasiswa untuk membangun koneksi akademik & sosial



Explore

Jelajahi jaringan sosialmu lewat Social Radar visualizer interaktif berbasis graf

Problem & Solusi

Mengapa kami develop CampusCircle?



Masalah yang Ada



Mahasiswa sulit menemukan teman lintas fakultas yang punya kesamaan akademik & minat



Social media umum (Instagram, LINE) tidak dirancang untuk koneksi berbasis kurikulum & komunitas kampus



Tidak ada cara efisien untuk memetakan siapa yang punya teman atau mata kuliah yang sama



Rekomendasi teman yang berguna butuh traversal multi-hop yang tidak bisa dilakukan SQL secara efisien



Solusi CampusCircle



Graph database Neo4j memetakan relasi mahasiswa–matkul–hobi secara native dan efisien



Sistem rekomendasi berbasis mutual connection: temukan siapa yang punya kesamaan 2–3 derajat



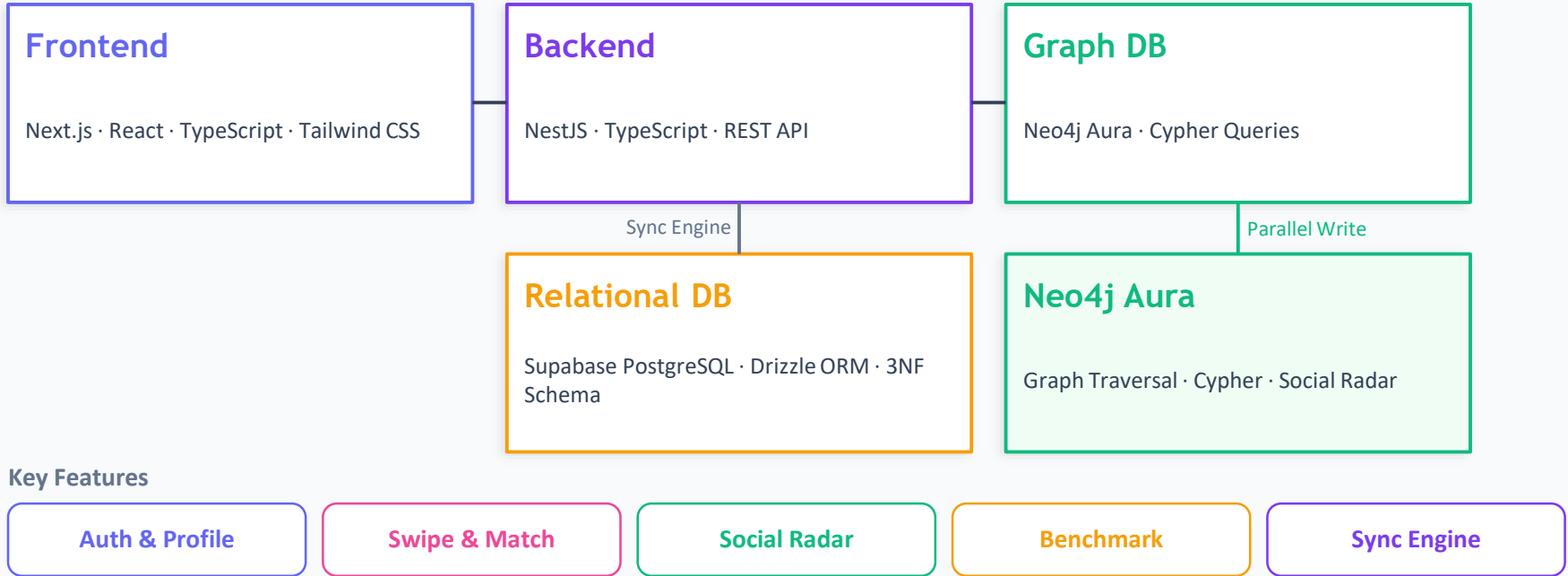
Social Radar interaktif memvisualisasikan jaringan sosial-akademik secara real-time



Dual-database (Neo4j + PostgreSQL) memberikan perbandingan performa langsung dan data integrity

Arsitektur Sistem

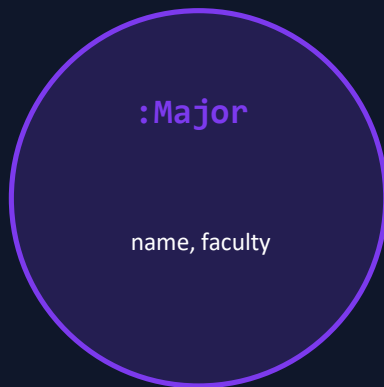
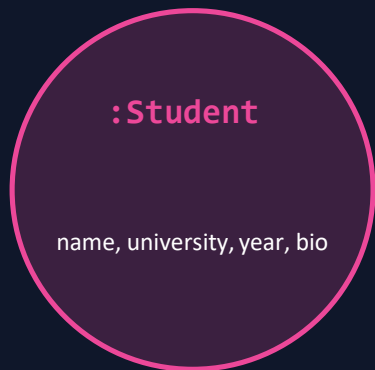
Dual-database architecture: PostgreSQL + Neo4j berjalan secara paralel



03

Data Modeling - Graph Schema (Neo4j)

Struktur node & relasi yang merepresentasikan jaringan sosial-akademik mahasiswa



Relationship Types

Relationship	From → To	Deskripsi
[:STUDIES]	:Student → :Major	Mahasiswa terdaftar di jurusan tertentu
[:TAKES]	:Student → :Course	Mahasiswa mengambil mata kuliah
[:LIKES]	:Student → :Hobby	Mahasiswa memiliki hobi/minat
[:MATCHES]	:Student → :Student	Kedua mahasiswa saling match (mutual like)
[:BLOCKS]	:Student → :Student	Mahasiswa memblokir pengguna lain

Data Modeling - Relational Schema (PostgreSQL)

Database dinormalisasi hingga Third Normal Form (3NF) untuk menghindari redundansi

users

id, full_name, username, password, phone_number, university, year, bio, major_id

majors

id, name, faculty

courses

id, code, name, faculty

hobbies

id, name, category

user_courses

user_id (FK→users), course_id (FK→courses)

user_hobbies

user_id (FK→users), hobby_id (FK→hobbies)

✦ user_courses & user_hobbies merupakan junction table (many-to-many) yang merepresentasikan relasi dalam bentuk tabel pivot — setara dengan edge dalam graf Neo4j.

Cypher vs SQL - Query Comparison

Query: Temukan mahasiswa yang berbagi matkul & hobi yang sama (Depth 1)

PostgreSQL (SQL)

```
SELECT DISTINCT u3.id, u3.full_name
FROM users u1
JOIN user_courses uc1 ON u1.id = uc1.user_id
JOIN user_courses uc2 ON uc1.course_id = uc2.course_id
  AND uc1.user_id != uc2.user_id
JOIN users u2 ON uc2.user_id = u2.id
JOIN user_courses uc3 ON u2.id = uc3.user_id
JOIN user_courses uc4 ON uc3.course_id = uc4.course_id
  AND uc3.user_id != uc4.user_id
JOIN users u3 ON uc4.user_id = u3.id
WHERE u1.username = 'mrbeast' AND u3.id != u1.id;
```

Deskripsi: Pada kedalaman 1-hop, performa SQL masih sangat bersaing dengan rata-rata eksekusi 64,97 ms. Pencarian indeks sederhana pada tabel pivot masih efisien karena relasi yang diperiksa belum bercabang banyak.

Neo4j (Cypher)

```
MATCH
  (s1:Student {name: "MrBeast"})
  -[:TAKES]->(c:Course)
  <-[:TAKES]-(s2:Student),
  (s1)-[:LIKES]->(h:Hobby)
  <-[:LIKES]-(s2)
RETURN DISTINCT s2.name;
```

Deskripsi: Neo4j menyelesaikan query sedikit lebih cepat dengan rata-rata 39,13 ms. Meskipun perbedaan performa belum drastis, penulisan sintaks Cypher jauh lebih ringkas (7 baris vs 14 baris SQL) tanpa membutuhkan JOIN eksplisit.

Cypher vs SQL - Query Comparison

Query: Temukan mahasiswa yang terhubung melalui teman sekelas (Depth 2 / 2-Hop)

PostgreSQL (SQL)

```
SELECT DISTINCT u2.id, u2.full_name
FROM users u1
JOIN user_courses uc1
  ON u1.id = uc1.user_id
JOIN user_courses uc2
  ON uc1.course_id = uc2.course_id
  AND uc1.user_id != uc2.user_id
JOIN users u2
  ON uc2.user_id = u2.id
JOIN user_hobbies uh1
  ON u1.id = uh1.user_id
JOIN user_hobbies uh2
  ON uh1.hobby_id = uh2.hobby_id
  AND uh2.user_id = u2.id
WHERE u1.username = 'mrbeast';
```

Membutuhkan 7 JOIN bertingkat hanya untuk menelusuri satu jenis relasi (mata kuliah). Membaca dan melakukan debugging pada query mulai menjadi sulit, meskipun performanya masih bisa ditoleransi (rata-rata ~97ms).

Neo4j (Cypher)

```
MATCH (s1:Student {name: "MrBeast"})
  -[:TAKES]->(:Course)<-[:TAKES]-(s2:Student)
  -[:TAKES]->(:Course)<-[:TAKES]-(s3:Student)
WHERE s1 <> s3
RETURN DISTINCT s3.name;
```

Di Cypher, kita hanya perlu menambahkan satu rantai visual `()-[]->()` lagi. Secara penulisan sangat elegan dan waktu eksekusinya tetap efisien tanpa membebani developer dengan logika tabel pivot.

Cypher vs SQL - Query Comparison

Query: Temukan koneksi sosial akademik yang dalam (Depth 3 / 3-Hop Course Chain)

PostgreSQL (SQL)

```
SELECT DISTINCT u4.id, u4.full_name
FROM users u1
JOIN user_courses uc1 ON u1.id = uc1.user_id
JOIN user_courses uc2 ON uc1.course_id = uc2.course_id
  AND uc1.user_id != uc2.user_id
JOIN users u2 ON uc2.user_id = u2.id
JOIN user_courses uc3 ON u2.id = uc3.user_id
JOIN user_courses uc4 ON uc3.course_id = uc4.course_id
  AND uc3.user_id != uc4.user_id
JOIN users u3 ON uc4.user_id = u3.id
JOIN user_courses uc5 ON u3.id = uc5.user_id
JOIN user_courses uc6 ON uc5.course_id = uc6.course_id
  AND uc5.user_id != uc6.user_id
JOIN users u4 ON uc6.user_id = u4.id
WHERE u1.username = 'mrbeast'
  AND u4.id NOT IN (u1.id, u2.id);
```

Query membengkak dengan 10 JOIN. Terjadi ledakan Cartesian product dari junction table yang masif. PostgreSQL memakan waktu rata-rata 16.101 ms (16,1 detik), membuktikan SQL tidak dirancang untuk traversal multi-hop ini.

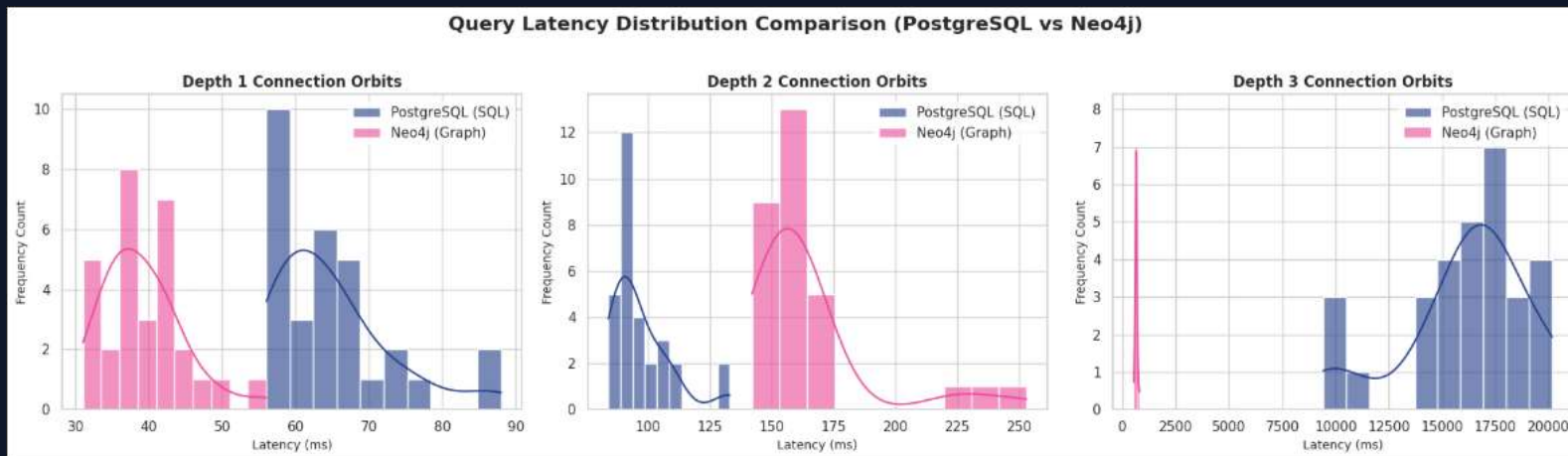
Neo4j (Cypher)

```
MATCH
  (s1:Student {name: "MrBeast"})
  -[:TAKES]->(c:Course)
  <-[:TAKES]-(s2:Student),
  (s1)-[:LIKES]->(h:Hobby)
  <-[:LIKES]-(s2)
RETURN DISTINCT s2.name;
```

Berkat mekanisme Index-Free Adjacency, Neo4j mengeksekusi traversal 3 derajat ini secara presisi dalam ~645 ms (~25x lebih cepat dari SQL). Pemangkasan jalur ganda (hop pruning) bekerja sempurna.

Benchmark Results

PostgreSQL vs Neo4j — 30 independent runs, 3 traversal depths



Depth 1

SQL: 64.97ms vs Neo4j: 39.13ms

🏆 Neo4j — 1.7× Neo4j lebih cepat

Depth 2

SQL: 96.87ms vs Neo4j: 164.43ms

🏆 SQL — 1.7× SQL unggul (simple joins cukup)

Depth 3

SQL: 16,101ms vs Neo4j: 645.60ms

🏆 Neo4j — ~25× Neo4j jauh lebih cepat!

Benchmark - Key Insights

Kesimpulan 30 run benchmark SQL vs Graph

● SQL Degradasi Eksponensial di Depth 3

PostgreSQL mencapai rata-rata ~16.1 detik di Depth 3. Ini disebabkan oleh multiple JOIN pada junction table `user_courses` dan `users` yang menghasilkan Cartesian product state space yang masif — semakin banyak hop, semakin besar ledakan data.

▣ Neo4j Skalabilitas Linear dengan Index-Free Adjacency

Neo4j hanya butuh ~645ms di Depth 3 — sekitar 25× lebih cepat dari SQL. Mekanisme Index-Free Adjacency memungkinkan traversal pointer langsung antar node tanpa perlu full table scan, ditambah WITH DISTINCT pruning untuk membuang jalur redundan.

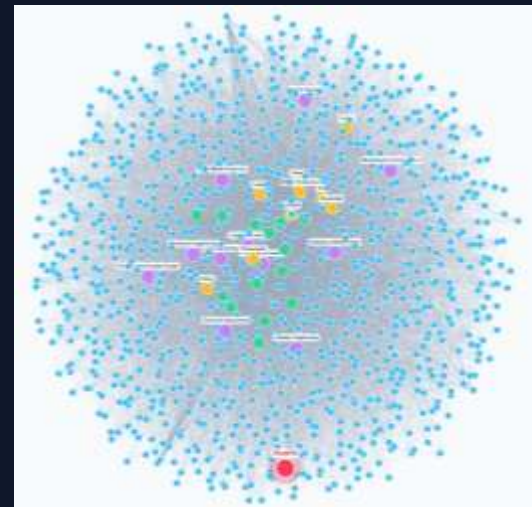
🏆 Sweet Spot: SQL Masih Kompetitif di Depth 1-2

Di Depth 1, SQL (64ms) vs Neo4j (39ms). Di Depth 2, SQL (97ms) vs Neo4j (164ms) — SQL justru lebih cepat karena simple index lookups sudah cukup. Neo4j unggul signifikan hanya saat traversal menjadi dalam dan kompleks. Ini membuktikan bahwa pilihan database harus disesuaikan dengan pola akses data yang dominan.

Kesimpulan: Graph Database (Neo4j) adalah pilihan tepat untuk workload berbasis traversal sosial multi-hop.

Social Radar Visualizer

Fitur unggulan CampusCircle — visualisasi jaringan sosial-akademik secara real-time



My Personal Orbit

Menampilkan node kamu + jurusan, matkul, dan hobi — dengan efek physics transitions yang smooth



Show Shared Sparks

Memperluas orbit ke teman sesama jurusan, sekelas, atau yang punya hobi sama

05 Lessons Learned & Trade-offs

Temuan, kejutan, dan pelajaran yang kami dapatkan selama pengembangan CampusCircle

01 Graph DB tidak unggul mutlak

Neo4j memang unggul di Depth 3+, tapi di Depth 1–2 PostgreSQL masih sangat kompetitif atau bahkan lebih cepat. Pemilihan DB harus didasarkan pada pola query yang paling sering diakses, bukan hanya asumsi.

02 Dual-database menambah kompleksitas operasional

Menjaga parity antara PostgreSQL dan Neo4j butuh sync engine yang handal. Satu operasi write harus direplikasi ke dua sistem, dan debugging menjadi lebih kompleks ketika terjadi inkonsistensi.

03 Cypher lebih ekspresif untuk relasi, tapi ada learning curve

Bagi yang baru belajar, Cypher terasa intuitive karena meniru pola visual graf (node)-[relasi]->(node). Tapi memahami kapan harus menggunakan MERGE vs CREATE, dan optimasi WITH DISTINCT untuk pruning, butuh waktu.

04 Index-Free Adjacency adalah game changer untuk deep traversal

Mekanisme Neo4j yang menyimpan pointer langsung ke tetangga (bukan via index) terbukti secara empiris dalam benchmark kami: Depth 3 yang butuh 16 detik di SQL, selesai dalam 645ms di Neo4j.

Individual Contributions

Rincian kontribusi setiap anggota kelompok — Kelompok Siang 11

Raja Avicenna Al-Kindi Vijasa

2406416352

- Inisiasi github repository
- Membuat rancangan awal CampusCircle

Toriq Fathoni Dezi

2406487115

- Dokumentasi laporan proposal
- Merancang database Neo4j

Danish Al Fayyadh Sunarta

2406416951

- pengembangan beberapa fitur tambahkan seperti algoritma swipe berdasarkan jumlah kecocokan dan page “who likes me?”
- Dokumentasi README.md, slide presentasi, dan laporan real-world case study

Alwahib Raffi Raihan

2406397630

- Pengembangan fitur message dan relasi swipe deck
- Fix beberapa problem yang muncul

Perry Tjahya

2406409965

- Implementasi arsitektur dual-database
- Membuat script untuk men-generate ribuan data sintetis untuk pengujian
- Benchmark Performa SQL vs GraphDB

Terima Kasih!

CampusCircle – Kelompok Siang 11

